
Sorting

- ❑ Bubble sort
- ❑ Insertion sort
- ❑ Selection sort
- ❑ Heap sort
- ❑ Merge sort
- ❑ Quicksort

Overview

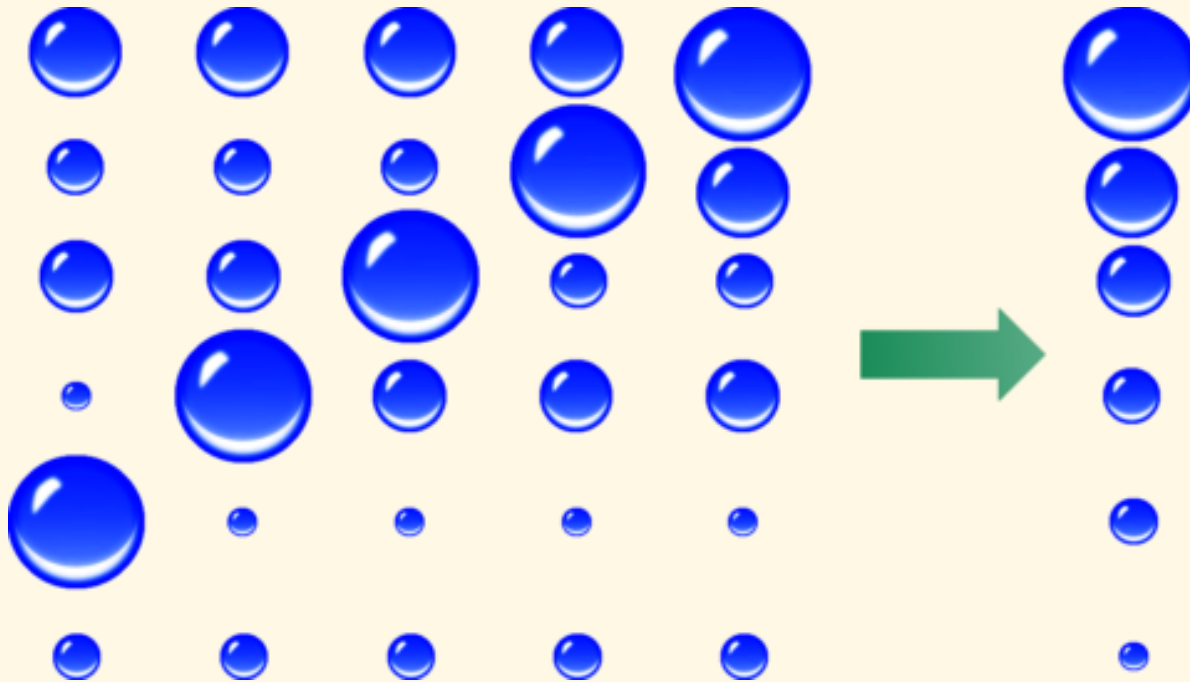
- ▶ Arranging data in a certain order
- ▶ Purposes:
 - ▶ Visualization
 - ▶ Searching
- ▶ Strategies to sort data
 - ▶ Let's imagine how we arrange cards or documents in order
- ▶ How to judge the efficiency of a sorting algorithm?
 - ▶ Time taken
 - ▶ Memory used
 - ▶ Data stored in an array or a linked list



Bubble sort

Overview

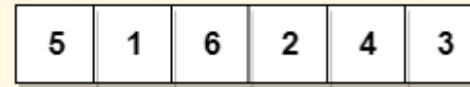
- ▶ Principle
 - ▶ Bigger elements bubble toward the end of the list



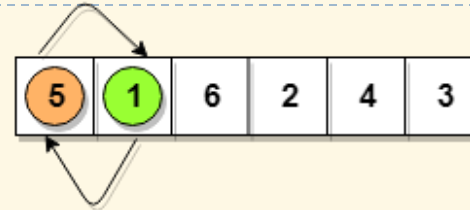
Algorithm

- ▶ Repeat until the array is sorted:
 - ▶ Step through the array
 - ▶ Compare each pair of **adjacent items**
 - ▶ Swap them if they are in the wrong order

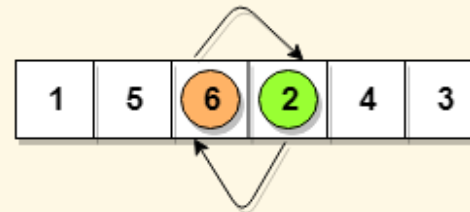
$5 > 1$
so interchange



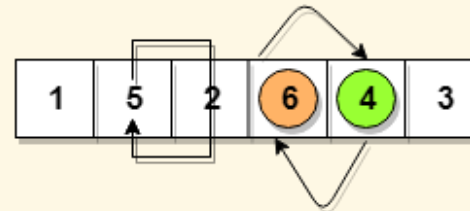
$5 < 6$
No swapping



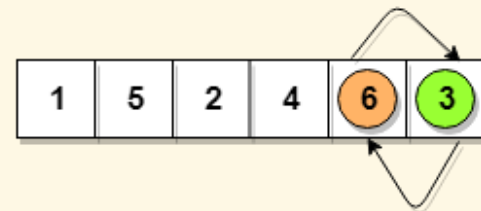
$6 > 2$
so interchange



$6 > 4$
so interchange



$6 > 3$
so interchange



This is
first
iteration

Do
similarly
for next
iterations
until the
array is
sorted

Naïve version (without termination checking)

```
void sortBubbleSimple(int* a, int n) {  
    int i, j;  
    for (i = n; i > 0; i--) {  
        for (j = 0; j < i - 1; j++) {  
            if (a[j] > a[j + 1])  
                swap(a, j, j + 1);  
        }  
    }  
}
```

With termination checking

```
void sortBubbleImproved(int* a, int n) {  
    int i, j, sorted = 0;  
    for (i = n; !sorted && i > 0; i--) {  
        sorted = 1;  
        for (j = 0; j < i - 1; j++) {  
            if (a[j] > a[j + 1]) {  
                swap(a, j, j + 1);  
                sorted = 0;  
            }  
        }  
    }  
}
```

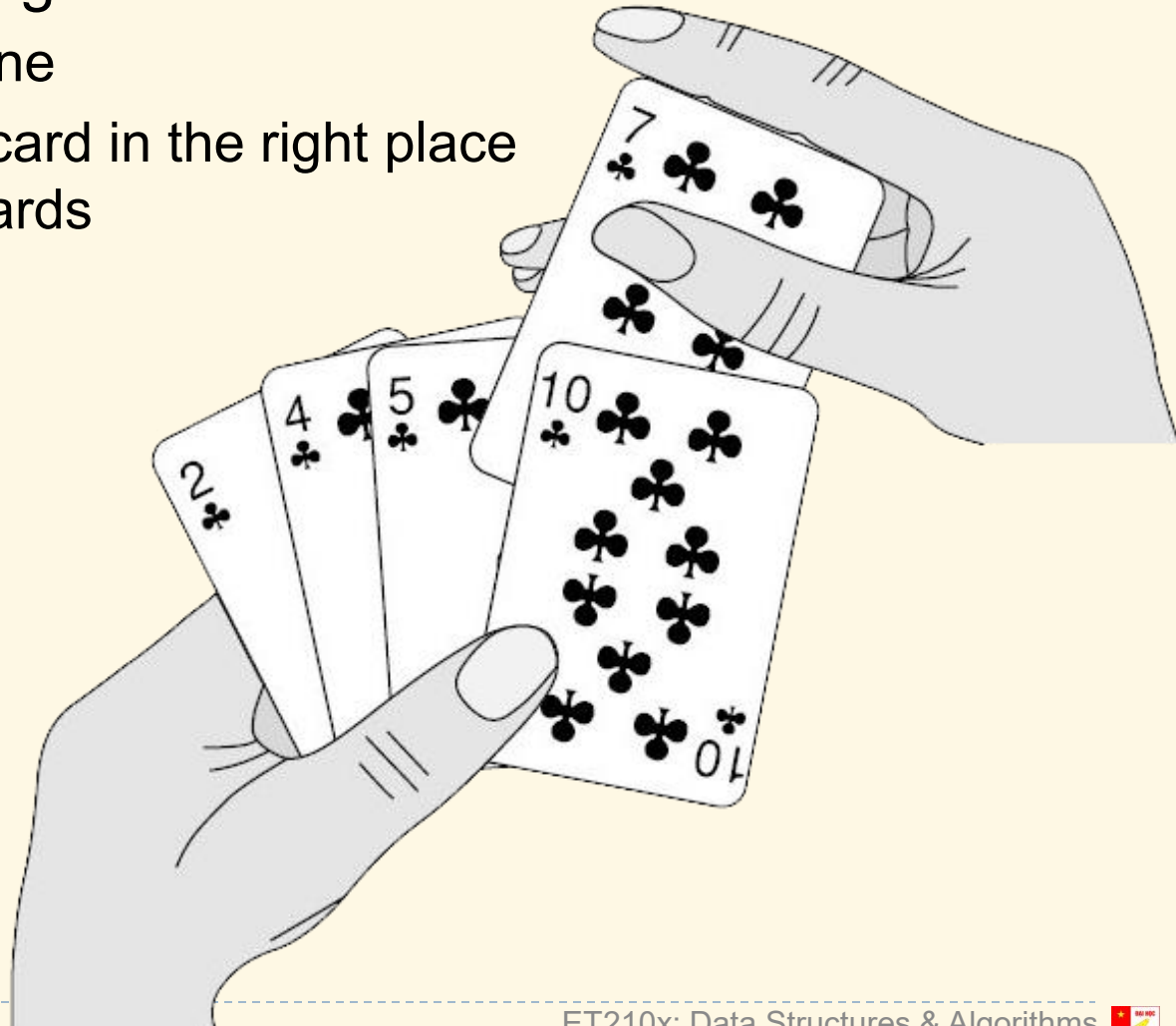
Complexity

- ▶ Computation:
 - ▶ Worst case: when array is in reversed order
 - ▶ Number of comparisons = no. of swaps = $n(n - 1)/2$
 - ▶ Complexity: $O(n^2)$
 - ▶ Average case: $O(n^2)$
 - ▶ Best case: when the array is already sorted
 - ▶ $O(n^2)$ for simple algorithm
 - ▶ $O(n)$ for improved algorithm
- ▶ Memory consumption: $O(1)$
- ▶ Generally not suitable in real applications
 - ▶ Good use cases
 - ▶ Data is almost sorted
 - ▶ Number of elements is small
- ▶ Can be implemented recursively

Insertion sort

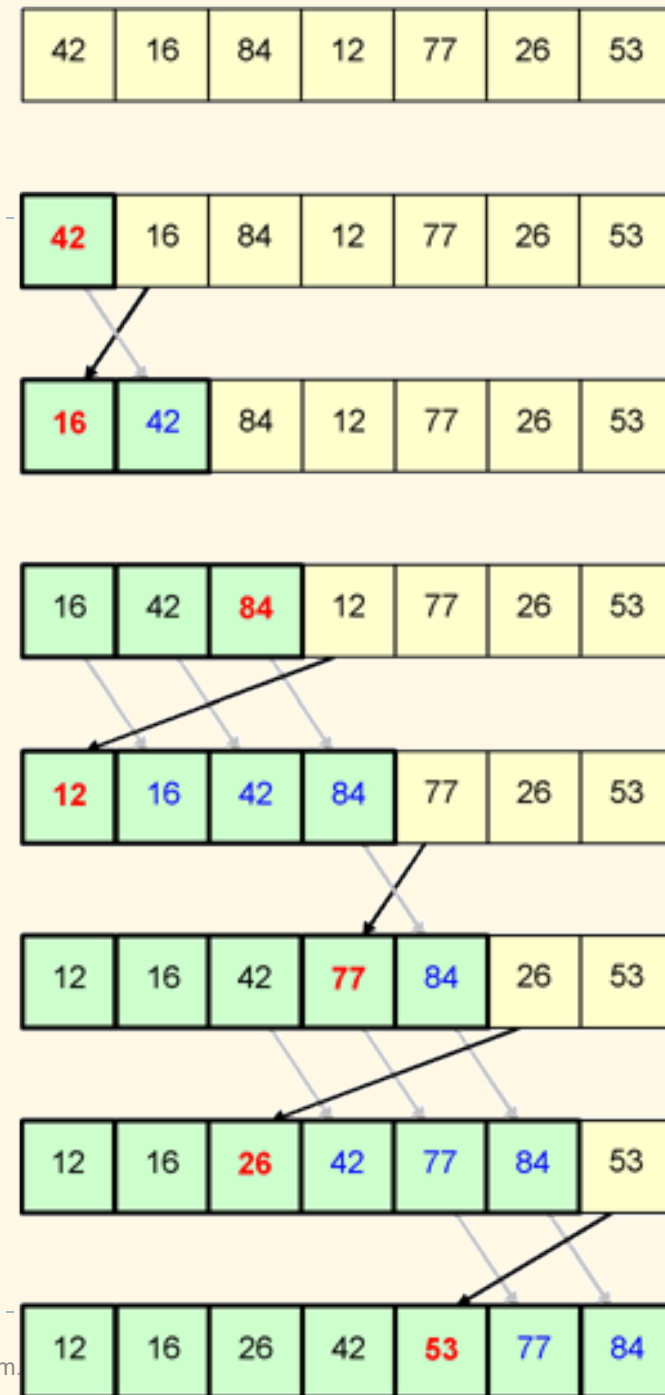
Overview

- ▶ Principle: Like sorting a hand of cards
 - ▶ Pick cards one by one
 - ▶ Each time, insert a card in the right place among the sorted cards



Algorithm

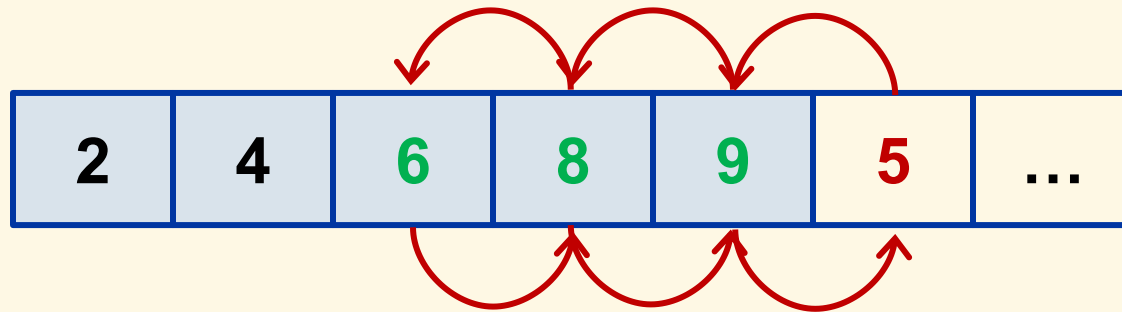
- ▶ Go through the array from left to right:
 - ▶ Pick the first element in the **unsorted part** (right part)
 - ▶ Find the right place for that element in the **sorted part** (left part)
 - ▶ Move (insert) the element to the found place
 - ▶ Expand the sorted part by 1 element, and shrink the unsorted part
- ▶ Attn: After the i^{th} iteration, the i left most elements are sorted part



Simple version

```
void sortInsertionSimple(int* a, int n) {  
    int i, j;  
    for (i = 1; i < n; i++) {  
        for (j = i; j > 0 && a[j] < a[j-1]; j--)  
            swap(a, j, j-1);  
    }  
}
```

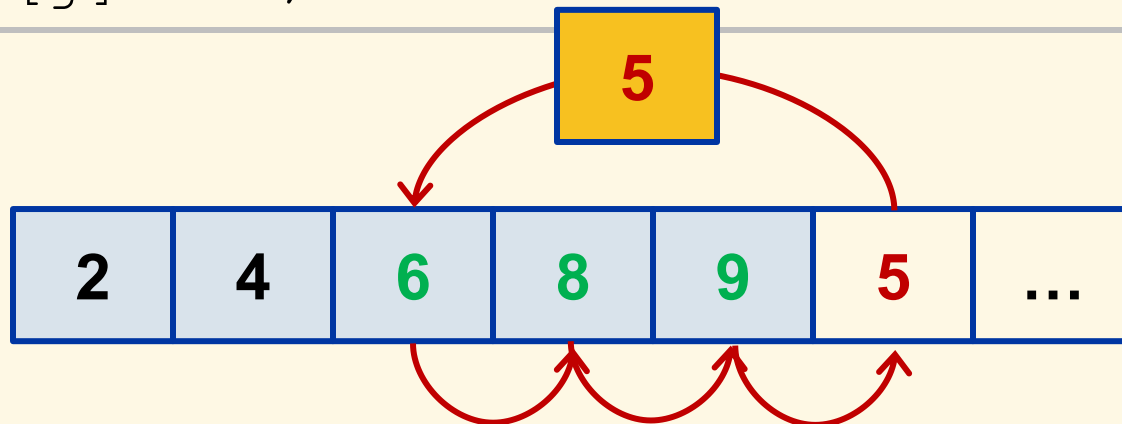
insertion



Better version

```
void sortInsertionImproved(int* a, int n) {  
    int i, j, t;  
    for (i = 1; i < n; i++) {  
        t = a[i];  
        for (j = i; j > 0 && t < a[j-1]; j--)  
            a[j] = a[j-1];  
        a[j] = t;  
    }  
}
```

insertion



Complexity

- ▶ Computation:
 - ▶ Worst case: when array is in reversed order
 - ▶ Number of comparisons = no. of moves = $n(n - 1)/2$
 - ▶ Complexity: $O(n^2)$
 - ▶ Average case: $O(n^2)$
 - ▶ Best case: when the array is already sorted
 - ▶ $O(n)$ for simple algorithm
 - ▶ $O(n)$ for improved algorithm
- ▶ Memory consumption: $O(1)$
- ▶ Can be implemented recursively

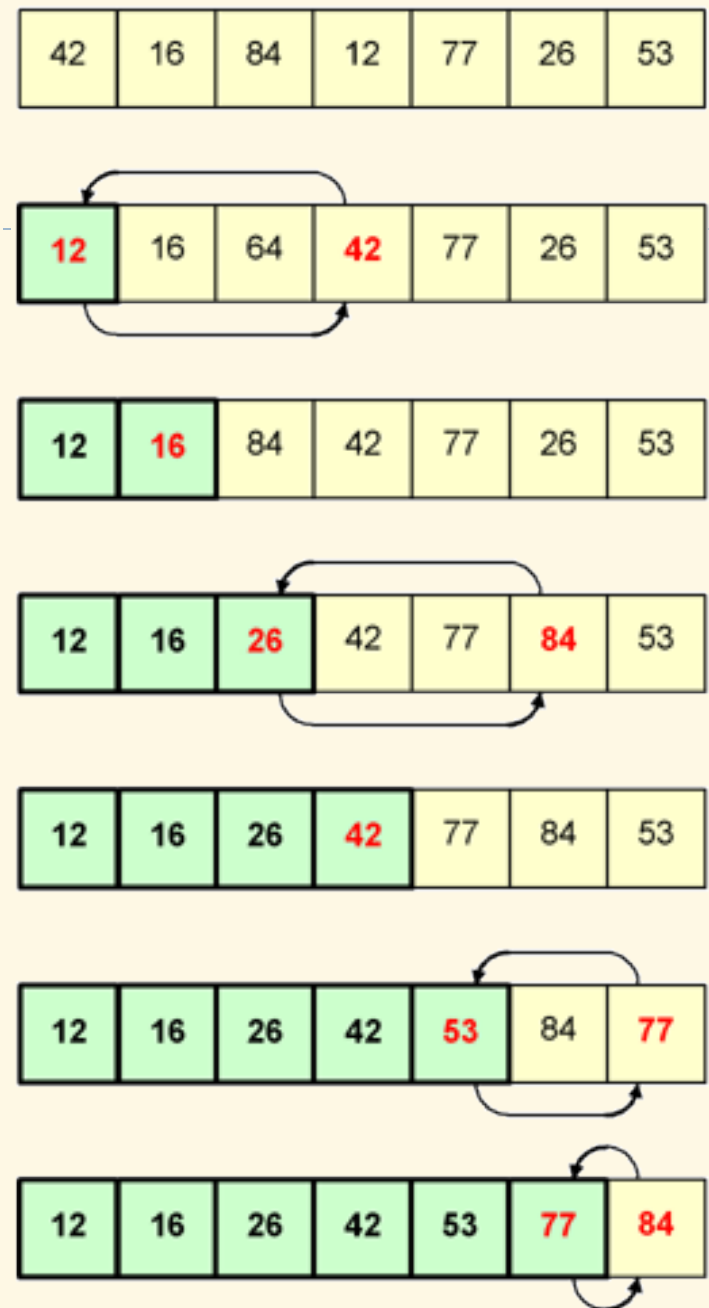
Selection sort

Overview

- ▶ For insertion sort, we pick the element first, then find the place to put it
- ▶ For selection sort, we do in the reverse way: find the place first, then find the element to put there

Algorithm

- ▶ Go from left to right of the array
 - ▶ Pick the **smallest element** from the **unsorted part**
 - ▶ Swap it with the first element in the unsorted part
 - ▶ Expand the **sorted part** by one
- ▶ Attn: After the i^{th} iteration, the i left most elements are sorted part



Implementation

```
void sortSelectionSimple(int* a, int n) {  
    int i, j;  
    for (i = 0; i < n-1; i++) {  
        for (j = i+1; j < n; j++) {  
            if (a[i] > a[j])  
                swap(a, i, j);  
        }  
    }  
}
```

Implementation

```
void sortSelectionImproved(int* a, int n) {  
    int i, j, mi;  
    for (i = 0; i < n-1; i++) {  
        for (mi = i, j = i+1; j < n; j++) {  
            if (a[j] < a[mi]) mi = j;  
        }  
  
        if (mi != i) swap(a, i, mi);  
    }  
}
```

Complexity

- ▶ Computation:
 - ▶ Worst case: when array is in reversed order
 - ▶ Number of comparisons: $n(n - 1)/2$
 - ▶ Number of swaps: n
 - ▶ Complexity: $O(n^2)$
 - ▶ Average case: $O(n^2)$
 - ▶ Best case: $O(n^2)$ when the array is already sorted
- ▶ Memory consumption: $O(1)$
- ▶ Can be implemented recursively

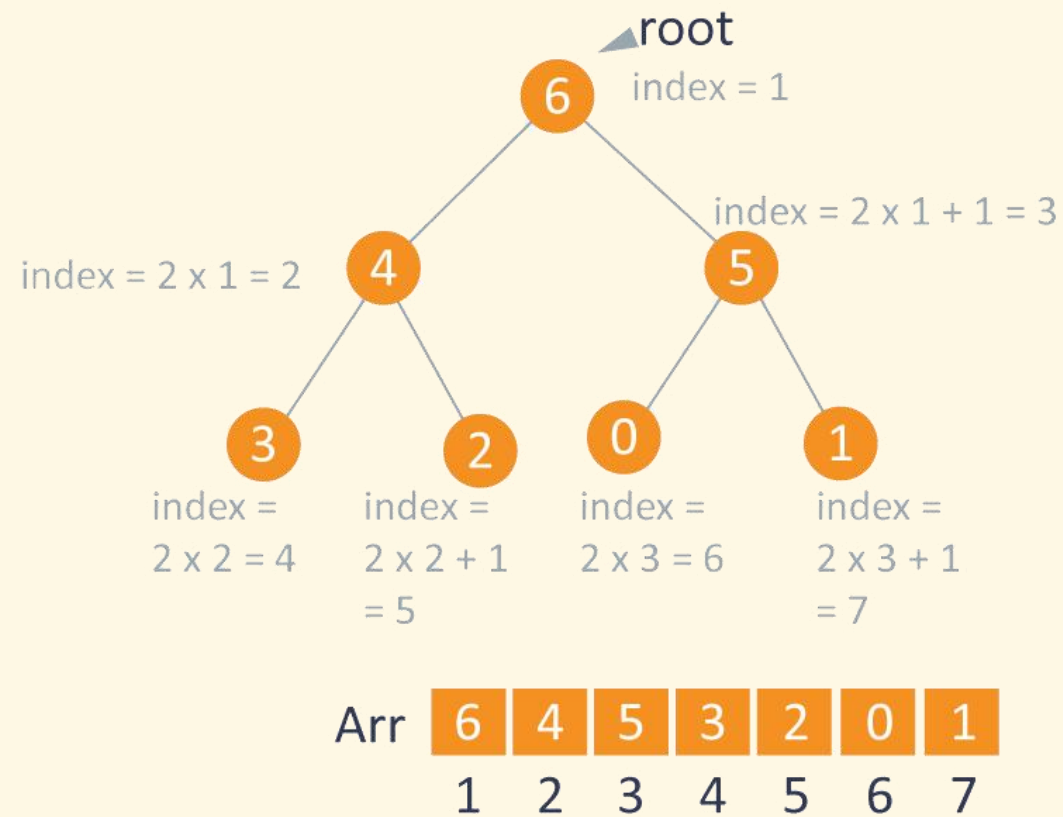
Exercises

- ▶ Rewrite the bubble, insertion, selection sort algorithms as recursive functions

Heap sort

Overview

- ▶ Based on a binary heap that helps to easily find the max (or min) elements
- ▶ Similar principle to Selection Sort, but the way to find the max elements is different
- ▶ Binary heap:
 - ▶ A complete binary tree
 - ▶ Two types: min heap, max heap
 - ▶ Value of each node in a max heap is greater than its children's
- ▶ No additional tree is needed since the tree is maintained by the array itself

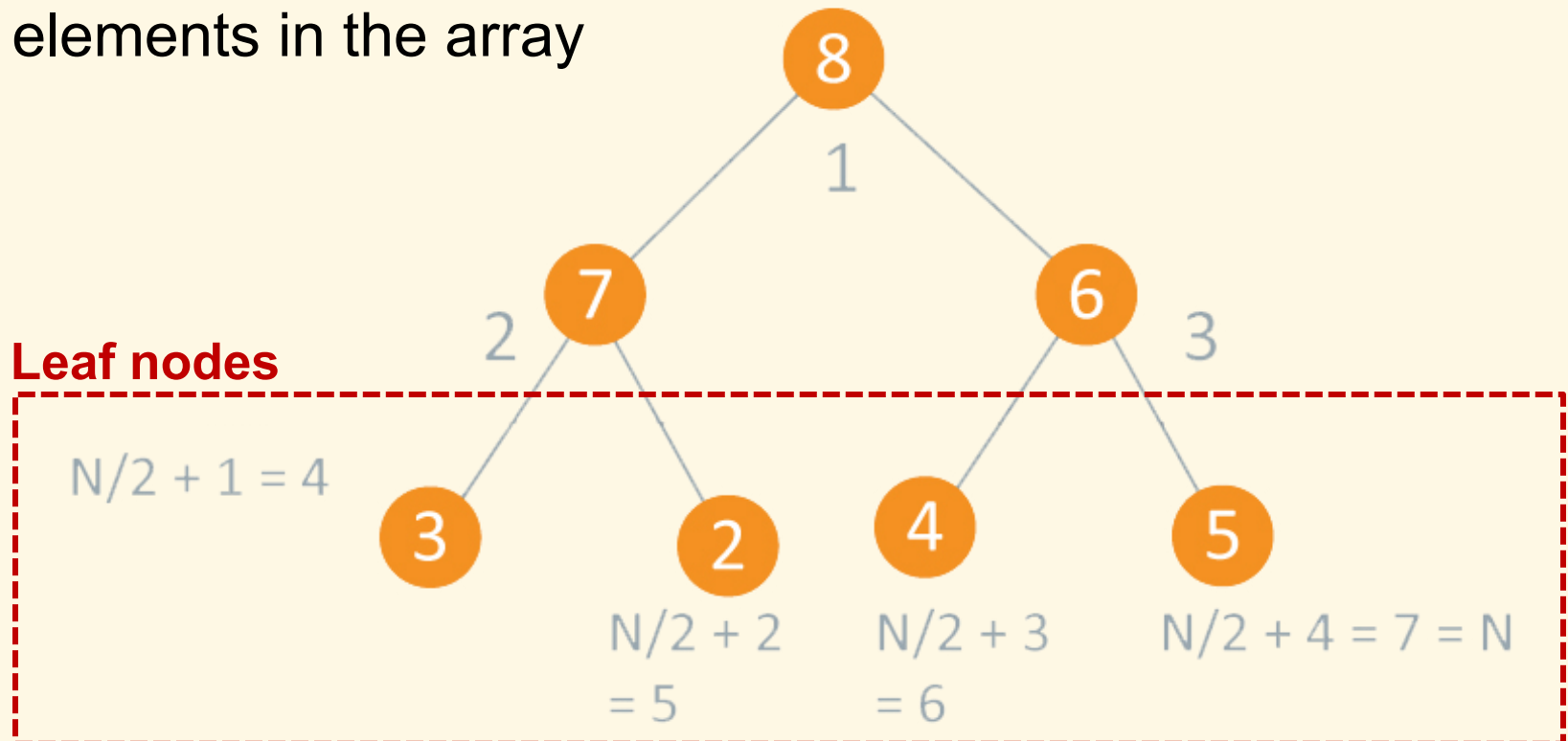


Algorithm

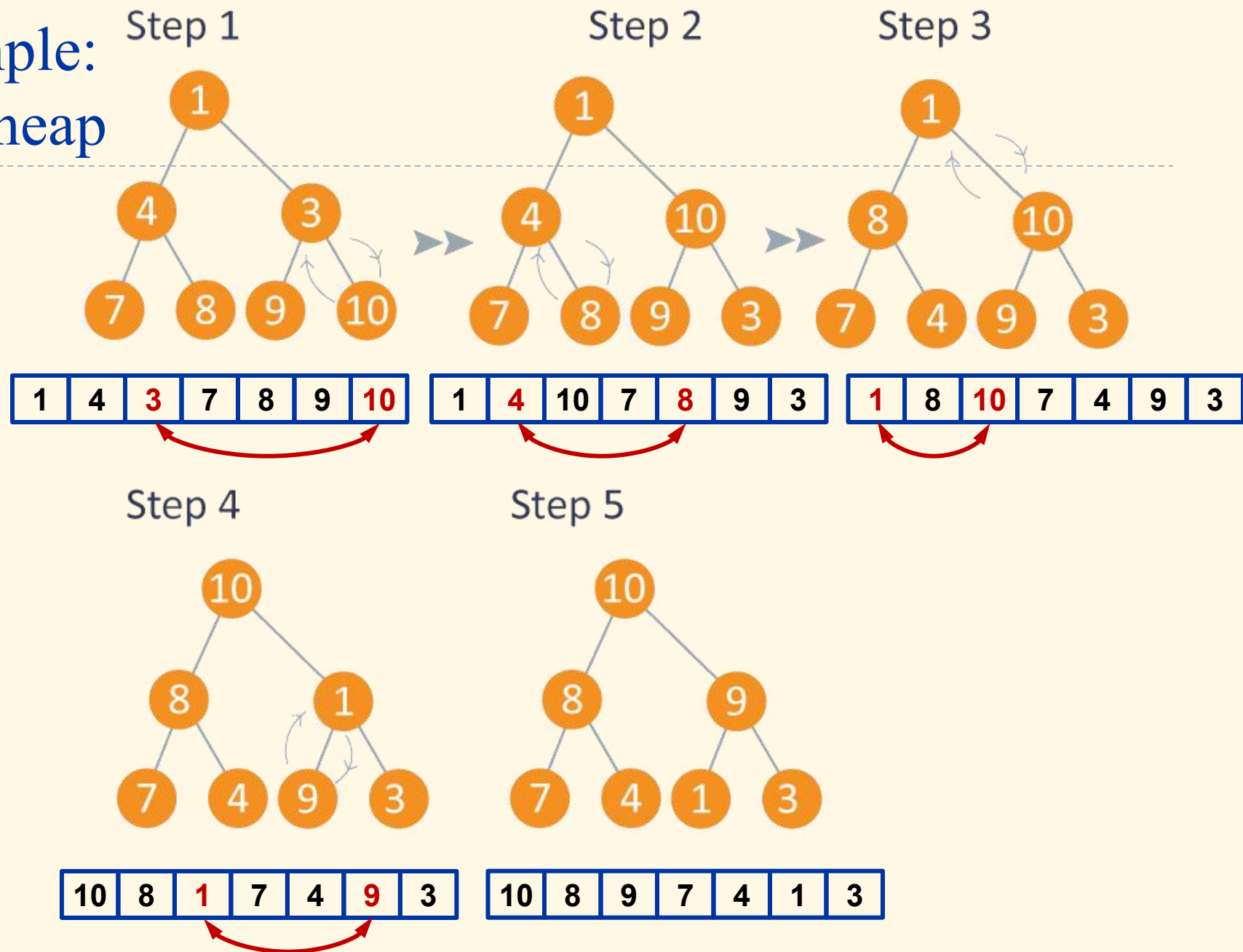
- ▶ Build a binary max heap
- ▶ Repeat until the array is sorted:
 - ▶ Swap the max (first) and the last elements in the array
 - ▶ Reduce the data size by 1
 - ▶ Correct the heap because of the root node's new value
- ▶ Attn:
 - ▶ In a max heap, the root element has the max value
 - ▶ The first element in the array corresponds to the root in the tree

Building max heap

- ▶ For each node, make sure that its children are smaller
- ▶ Check from lower nodes up to the root
- ▶ No need to check leaf nodes → Only check the first $n/2$ elements in the array

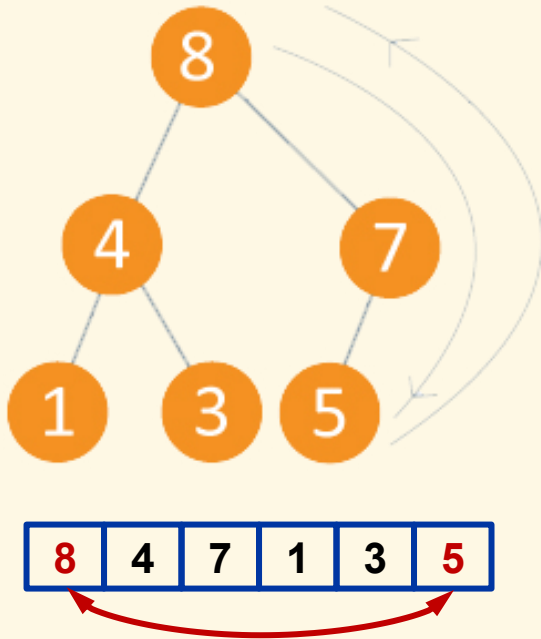


Example: Max heap

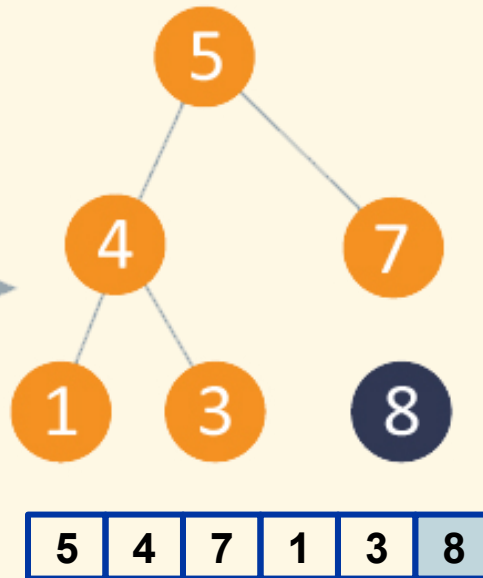


Example: Heap sort

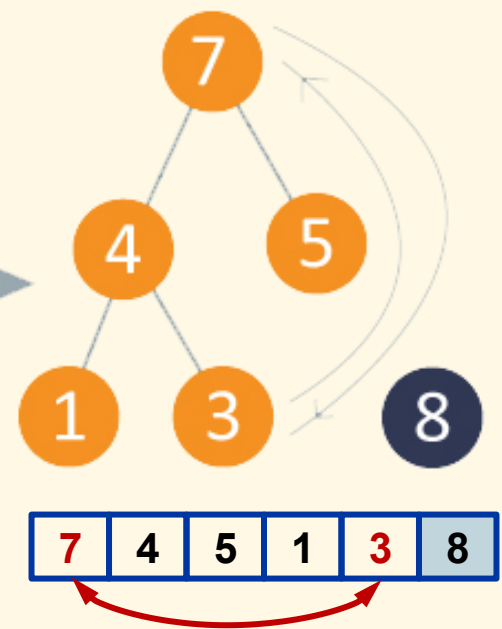
Step 1: Max heap



Step 2: Arrange

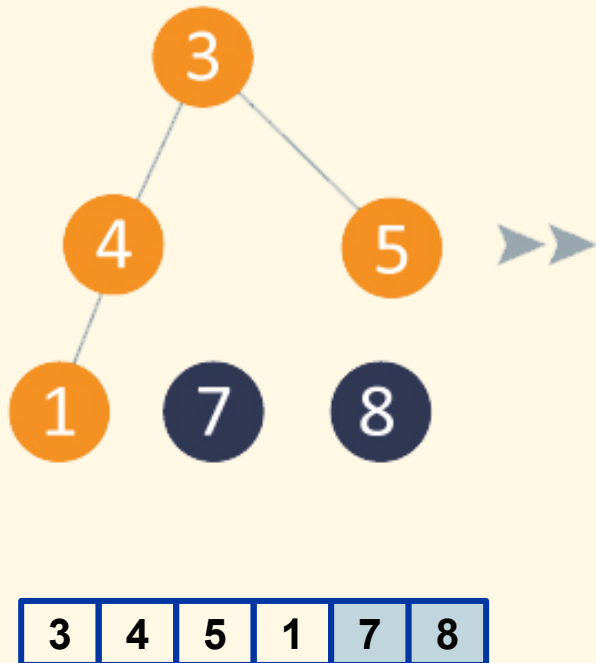


Step 3: Max heap

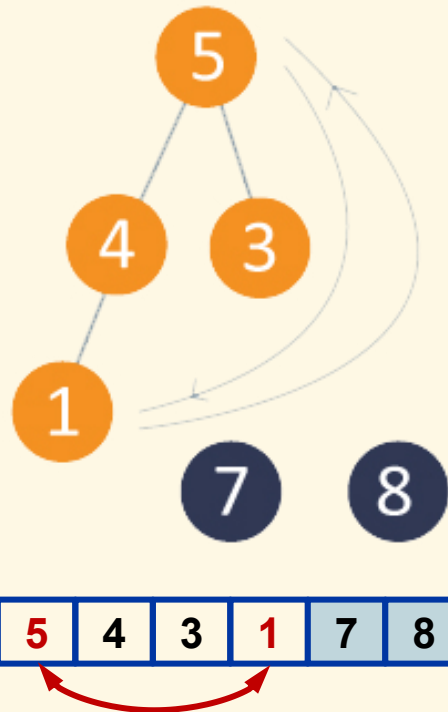


Example: Heap sort (*cont'd*)

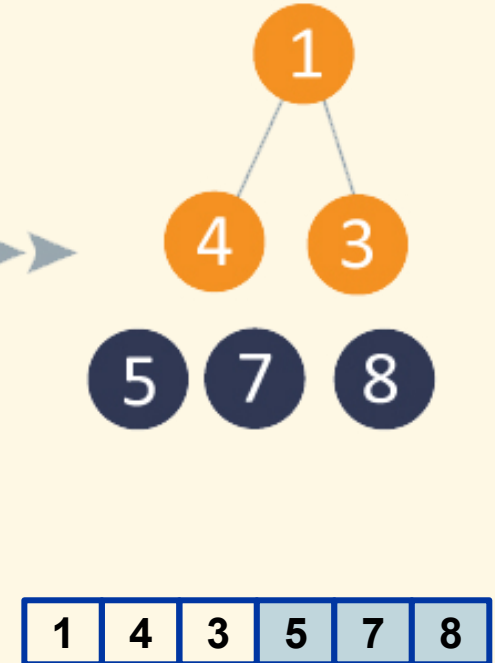
Step 4: Arrange



Step 5: Max heap

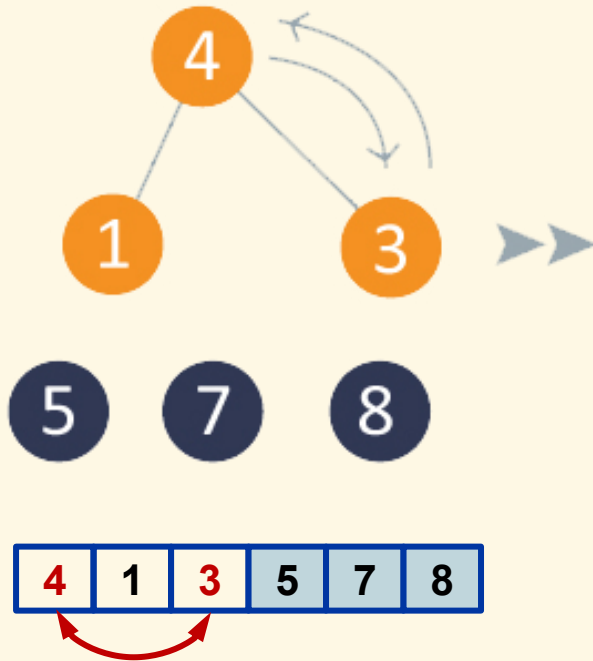


Step 6: Arrange

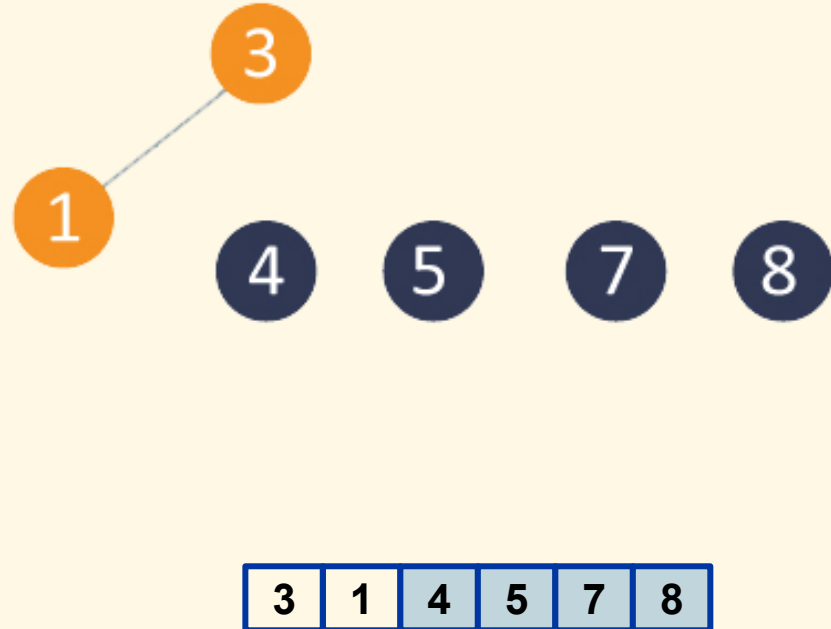


Example: Heap sort (*cont'd*)

Step 7: Max heap

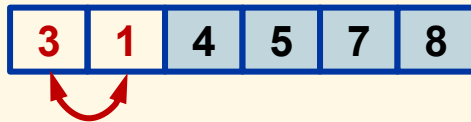


Step 8: Arrange

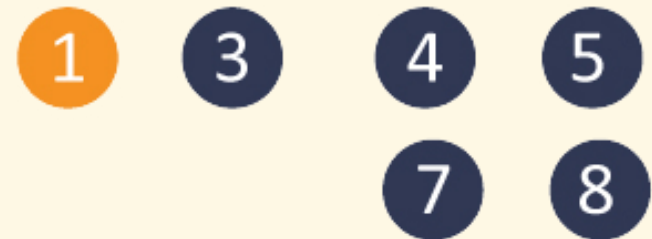


Example: Heap sort (*cont'd*)

Step 9: Max heap



Step 10: Arrange



Implementation: Correct heap node

```
void correctHeapNode(int* a, int n, int i) {  
    int l = i*2 + 1,  
        r = l + 1,  
        m = i;  
    if (l < n && a[l] > a[m]) m = l;  
    if (r < n && a[r] > a[m]) m = r;  
  
    if (m != i) {  
        swap(a, i, m);  
        correctHeapNode(a, n, m);  
    }  
}
```

Implementation: Max heap

```
void buildHeap(int* a, int n) {  
    for (int i = n/2; i >= 0; i--)  
        correctHeapNode(a, n, i);  
}
```


Implementation: Heap sort

```
void sortHeap(int* a, int n) {  
    buildHeap(a, n);  
  
    for (int i = n; i > 1; i--) {  
        swap(a, 0, i-1);  
        correctHeapNode(a, i-1, 0);  
    }  
}
```

Complexity

▶ Computation

- ▶ Correct node: $O(\log n)$
- ▶ Build heap: $O(n \log n)$
- ▶ Main loop: $O(n \log n)$
- ▶ Overall: $O(n \log n)$

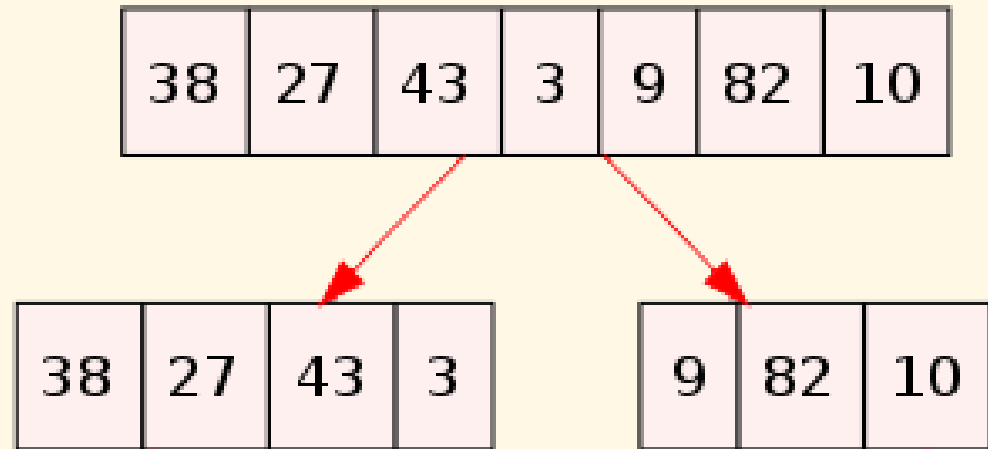
▶ Memory (stack)

- ▶ Correct node: $O(\log n)$
- ▶ Build heap: $O(\log n)$
- ▶ Main loop: $O(\log n)$
- ▶ Overall: $O(\log n)$

Merge sort

Overview

- ▶ Initial idea:
 - ▶ Bubble, insertion, selection sorts all have complexity of $O(n^2)$
 - ➔ The computational time grows quickly when n is large
 - ➔ Subdivide the problem into smaller tasks
- ▶ Divide and conquer approach (recursive implementation)
 - ▶ Divide into two parts
 - ▶ Sort each parts separately
 - ▶ Merge the two sorted parts



Algorithm

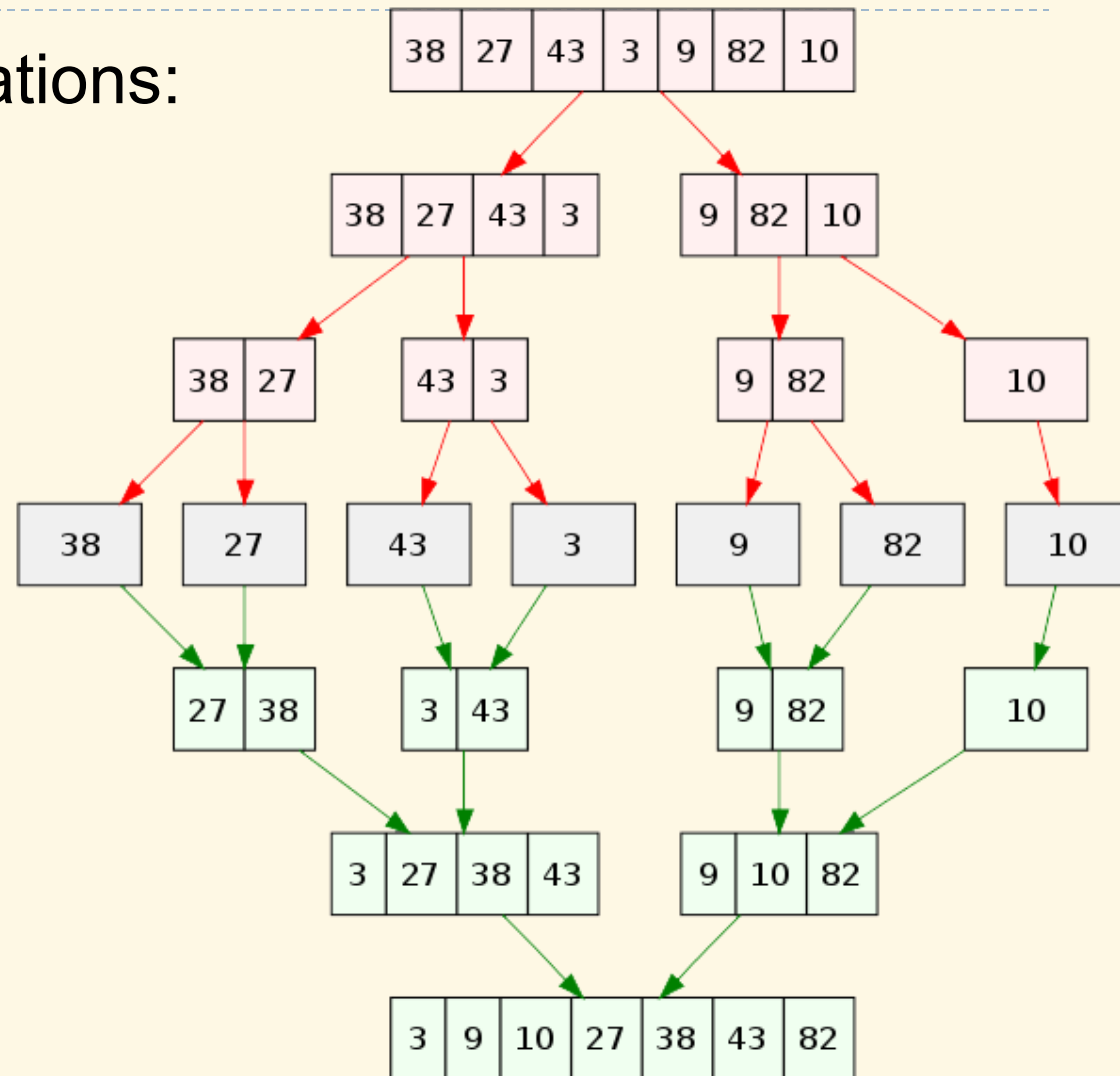
- ▶ Based on two operations:

- ▶ Division

- ▶ Divide continuously until the arrays become elementary (with 1 item)

- ▶ Merging

- ▶ Combine two sorted arrays



Implementation

```
void sortMergePart(int* a, int l, int r) {  
    int m = (l + r) / 2;  
    if (m > l) sortMergePart(a, l, m);  
    if (m + 1 < r) sortMergePart(a, m + 1, r);  
    mergePartsXXX(a, l, m, r);  
}
```

```
void sortMerge(int* a, int n) {  
    sortMergePart(a, 0, n-1);  
}
```

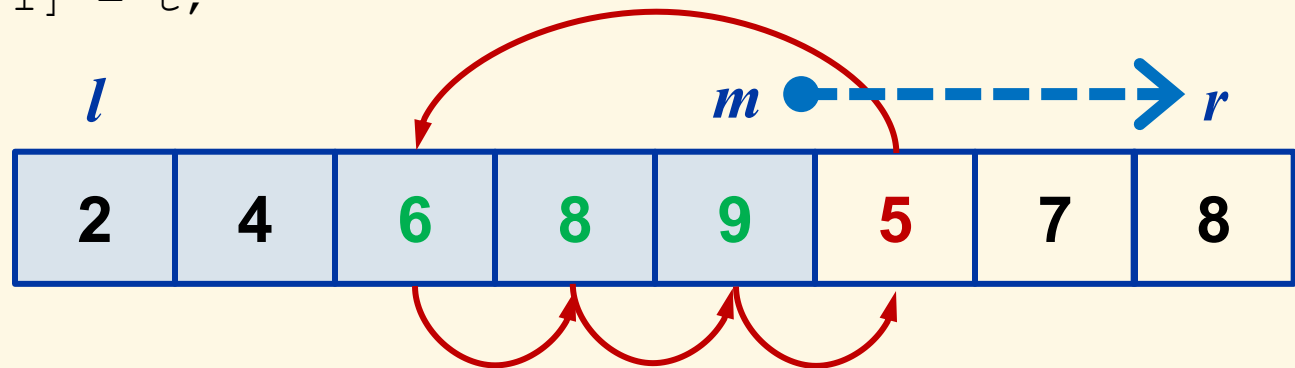
Merge arrays using auxiliary space (standard)

```
void mergeParts_aux(int* a, int l, int m, int r) {
    int pn = m - l + 1, qn = r - m;
    int *p = (int*)malloc(pn * sizeof(int)),
        *q = (int*)malloc(qn * sizeof(int));
    memcpy(p, a + l, pn * sizeof(int));
    memcpy(q, a + m + 1, qn * sizeof(int));
    for (int i = 0, j = 0; i < pn || j < qn; ) {
        if ((i == pn) || (j < qn && p[i] > q[j])) {
            a[l + i + j] = q[j];
            j++;
        } else {
            a[l + i + j] = p[i];
            i++;
        }
    }
    free(p);
    free(q);
}
```

Merge arrays without auxiliary space (in place)

- ▶ Since the two arrays are consecutive

```
void mergeParts_inPlace(int* a, int l, int m, int r) {  
    int i, t;  
    for (; m < r; m++) {  
        t = a[m+1];  
        for (i = m; i >= l && a[i] > t; i--)  
            a[i+1] = a[i];  
        a[i+1] = t;  
    }  
}
```



Complexity

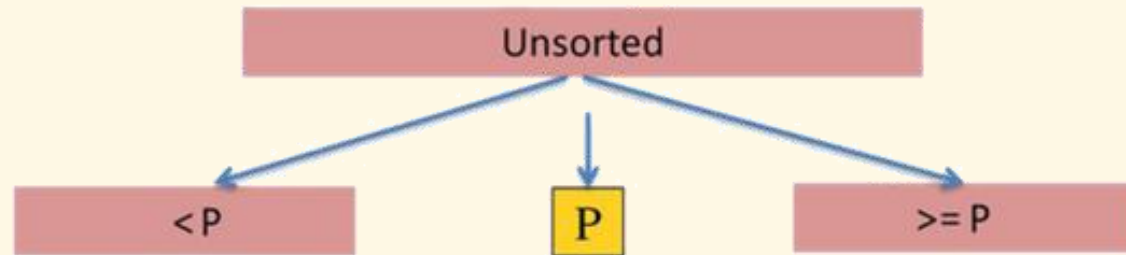
Complexity	Standard method	In-place method
Computation	$O(n \log n)$	$O(n^2 \log n)$
- Division	$O(\log n)$	$O(\log n)$
- Merging	$O(n)$	$O(n^2)$
Memory	$O(n)$	$O(\log n)$
- Division (stack)	$O(\log n)$	$O(\log n)$
- Merging (heap)	$O(n)$	$O(1)$

Quicksort

Overview

- ▶ Divide and conquer approach
- ▶ Principle:
 - ▶ Partition the data into two smaller sets using a threshold value (called pivot)
 - ▶ Do the same with the two subsets

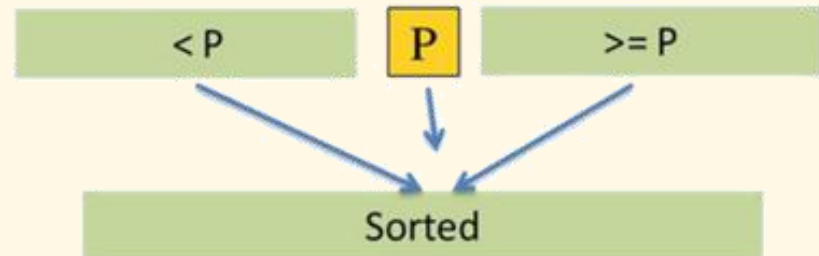
Divide: Pick a pivot, partition into groups



Conquer: Return array when length ≤ 1



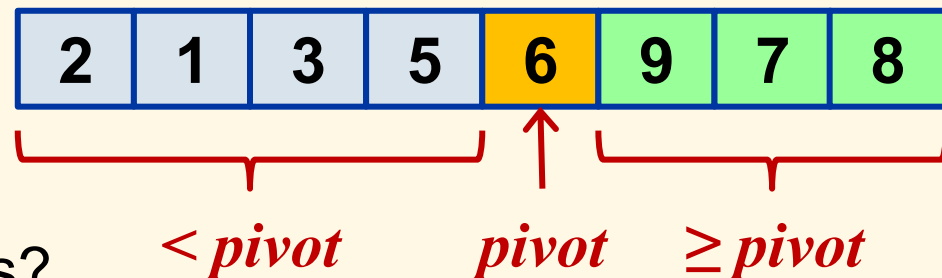
Combine: Combine sorted partitions and pivot



Algorithm

- ▶ If the array size is greater than 1
 - ▶ Pick a pivot element
 - ▶ Partition the array so that
 - ▶ Elements $<$ the pivot go to the left
 - ▶ Elements $\geq$ the pivot go to the right
 - ▶ Pivot is in between
 - ▶ Do the same with the left part
 - ▶ Do the same with the right part

- ▶ The problems:
 - ▶ How to partition?
 - ▶ How to choose good pivots?



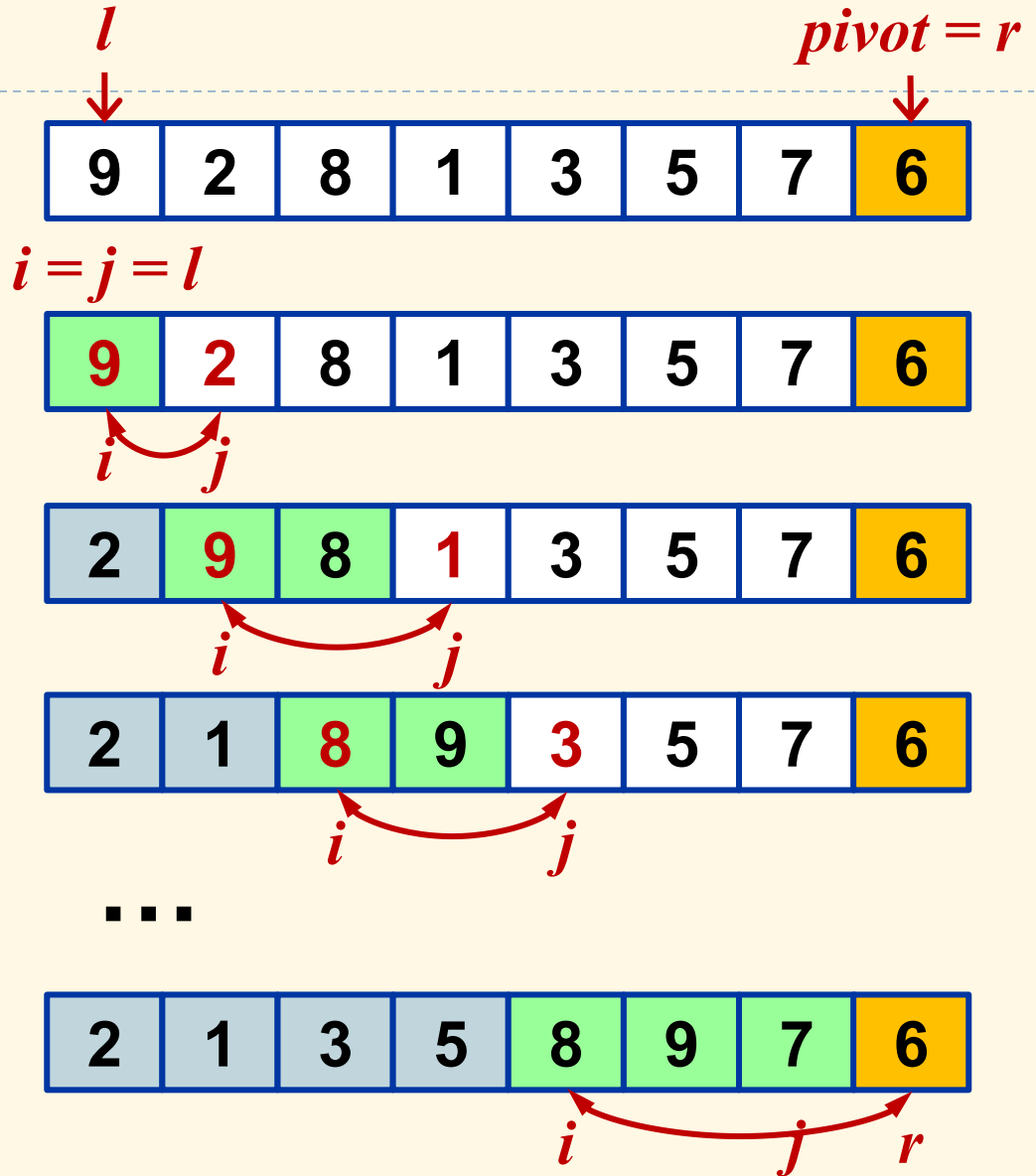
Implementation: Quicksort

```
void quicksortRange(int* a, int l, int r) {  
    if (l < r) {  
        int pi = qsPartitionXXX(a, l, r);  
        quicksortRange(a, l, pi - 1);  
        quicksortRange(a, pi + 1, r);  
    }  
}
```

```
void quicksort(int* a, int n) {  
    quicksortRange(a, 0, n - 1);  
}
```

Lomuto partition

- ▶ Pick last element as pivot
- ▶ i : index of the first element $\geq$ pivot; starts with l
- ▶ Repeat until $j = r$
 - ▶ j : find element $<$ pivot
 - ▶ Swap elements at i and j
 - ▶ Increase i
- ▶ Swap elements at i and r

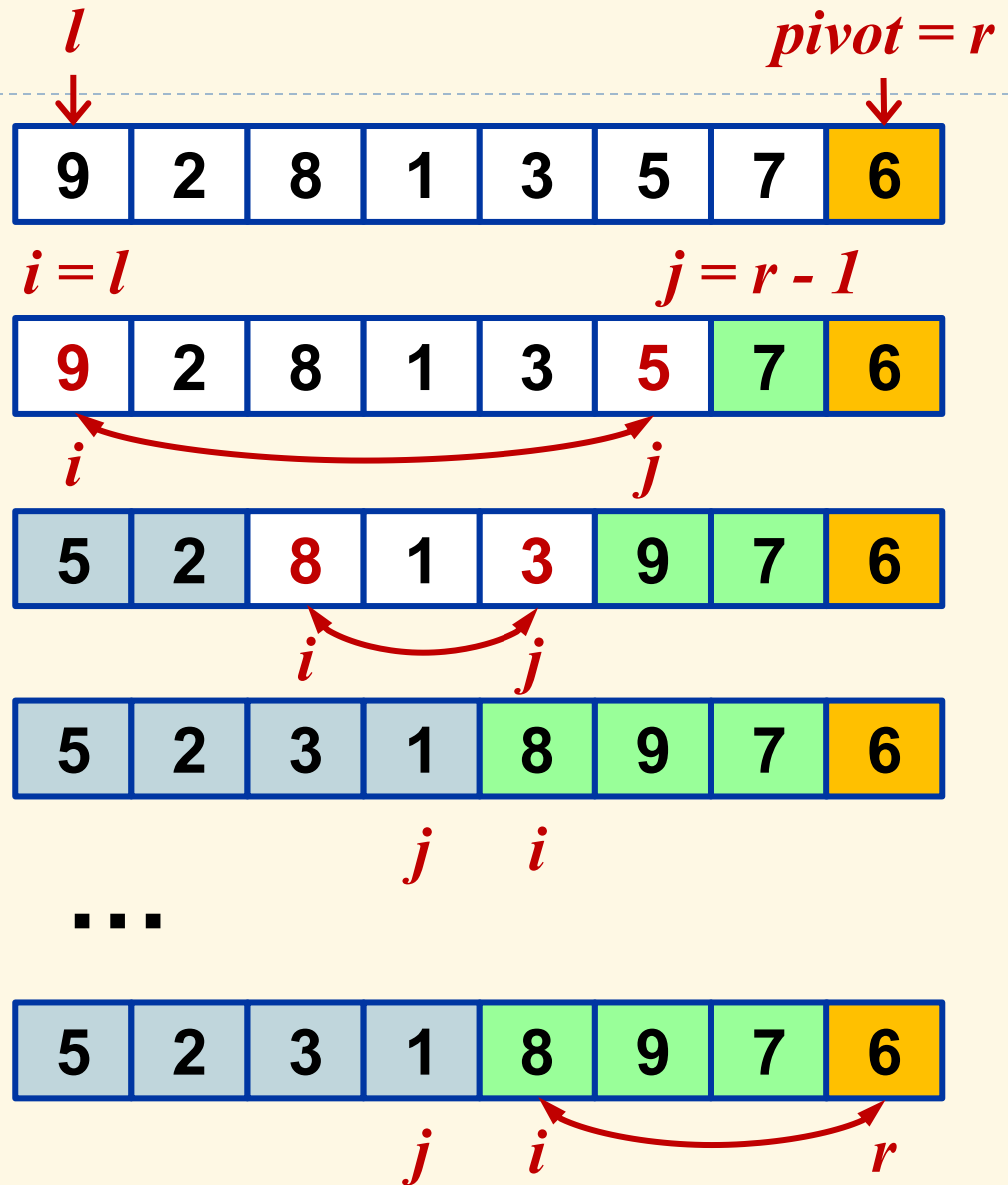


Implementation: Lomuto partition

```
int qsPartitionLomuto(int* a, int l, int r) {  
    int pivot = a[r];  
    int i, j;  
  
    for (i = l, j = l; j < r; j++) {  
        if (a[j] < pivot)  
            swap(a, i++, j);  
    }  
  
    swap(a, i, r);  
    return i;  
}
```

Hoare partition

- ▶ Pick last element as pivot
- ▶ i and j run from two ends
- ▶ Repeat until $i \geq j$
 - ▶ i : find element $\geq$ pivot forward
 - ▶ j : find element $<$ pivot backward
 - ▶ Swap elements at i and j
- ▶ Swap elements at i and r



Implementation: Hoare partition

```
int qsPartitionHoare(int* a, int l, int r) {  
    int pivot = a[r];  
    int i, j;  
  
    for (i = l, j = r - 1; i < j;) {  
        for (; a[i] < pivot && i < j; i++);  
        for (; a[j] >= pivot && i < j; j--);  
        swap(a, i, j);  
    }  
  
    if (a[r] > a[i]) return r;  
  
    swap(a, i, r);  
    return i;  
}
```

Edge case: When the chosen pivot has the greatest value, keep it as the last element.

Complexity

Complexity	Worst case	Best case	Average case
Computation	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
- Partition	$O(n)$	$O(n)$	$O(n)$
- Recursion	$O(n)$	$O(\log n)$	$O(\log n)$
Memory (stack)	$O(n)$	$O(\log n)$	$O(\log n)$
- Partition	$O(1)$	$O(1)$	$O(1)$
- Recursion	$O(n)$	$O(\log n)$	$O(\log n)$

- ▶ Worst case: when the algorithm always pick the smallest or largest element as pivot

Good strategy to choose pivots?

- ▶ Any element can be chosen as pivot (no need to be last/first one)
- ▶ Pivot being first or last element leads to worst case when the array is already sorted
- ▶ Other strategies:
 - ▶ Middle element
 - ▶ Random element
 - ▶ Median of {first, last, middle}

Summary

Complexity

Algorithm	Worst case	Best case	Average case	Memory
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$
Merge (std)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Merge (in place)	$O(n^2 \log n)$	$O(n^2 \log n)$	$O(n^2 \log n)$	$O(\log n)$
Quicksort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$

- ▶ Quicksort is considered as best choice for most of cases in practice

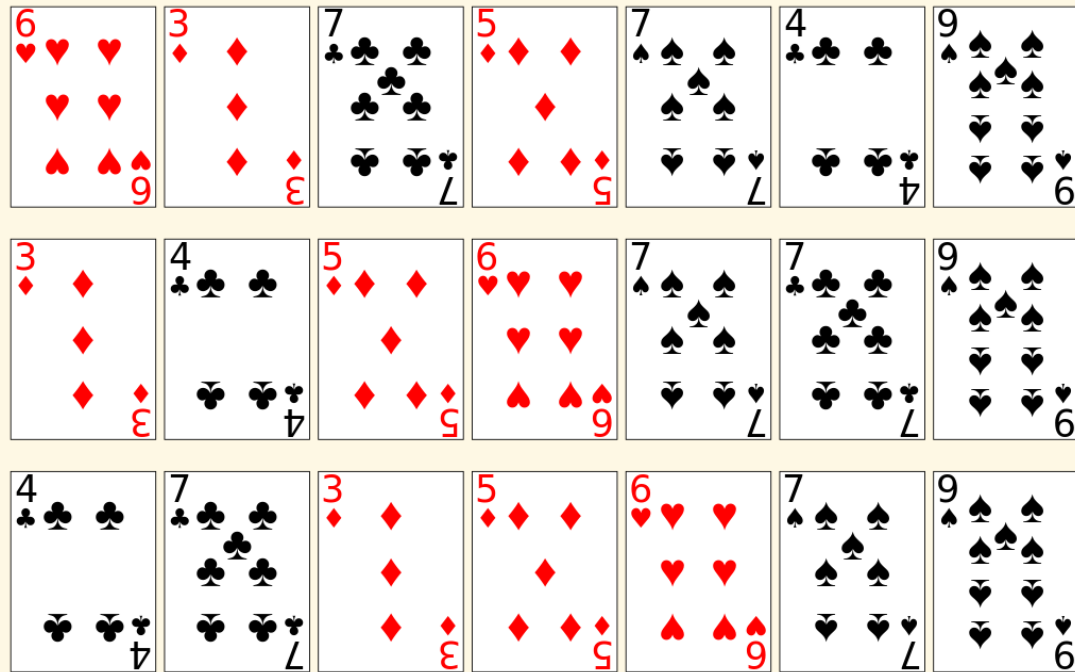
Other considerations

Algorithm	Recursive?	Parallel processors?	Parallel disks?	Cache efficient?
Bubble	—	—	—	—
Insertion	—	—	—	—
Selection	—	—	—	—
Heap	—	—	—	—
Merge (std)	+	+	+	+
Merge (in place)	+	+	+	+
Quicksort	+	+	—	+

+ good — bad

Sort stability

- ▶ A sorting algorithm is stable if items with same key preserve their order in the output as they were in the input
- ▶ Stable algorithms by nature: bubble, insertion, merge
- ▶ Unstable algorithms: selection, heap, quicksort
- ▶ Unstable algorithms can be made stable by
 - ▶ Adding original order into account of the comparison



Application example of sorting a list using primary and secondary keys: first sorting the cards by rank (using any sort), then doing a stable sort by suit

Further technical discussion

- ▶ How to avoid swapping large array elements?
- ▶ How to generalize data type and sort order?
- ▶ Which algorithms can be implemented with linked lists?
- ▶ Why don't we partition into more parts for divide-and-conquer algorithms?
- ▶ Other sorting algorithms
 - ▶ Bucket sort, shuffle sort, comb sort, Shell sort,...
 - ▶ Hybrid sorts
- ▶ Other kinds of sorting:
 - ▶ Sorting into groups (e.g., Dutch national flag problem)

ANSI C `qsort()` function

► Definition

```
void qsort(void* base, size_t num, size_t size,  
           int (*compare)(const void*, const void*));
```

► Example

```
int strorder(const void* a, const void* b) {  
    const char* sa = *(const char**)a,  
               * sb = *(const char**)b;  
    return strcmp(sa, sb);  
}
```

```
const char* s[] = {"abc", "xyz", ...};  
const int n = sizeof(s) / sizeof(s[0]);  
qsort(s, n, sizeof(s[0]), strorder);
```